Technische Hochschule Ingolstadt

Fakultät für Elektrotechnik und Informatik

Zukunft in Bewegung

Die Prozessicht

Dokumenthistorie

Autor: Prof. Dr. Bernd Hafenrichter

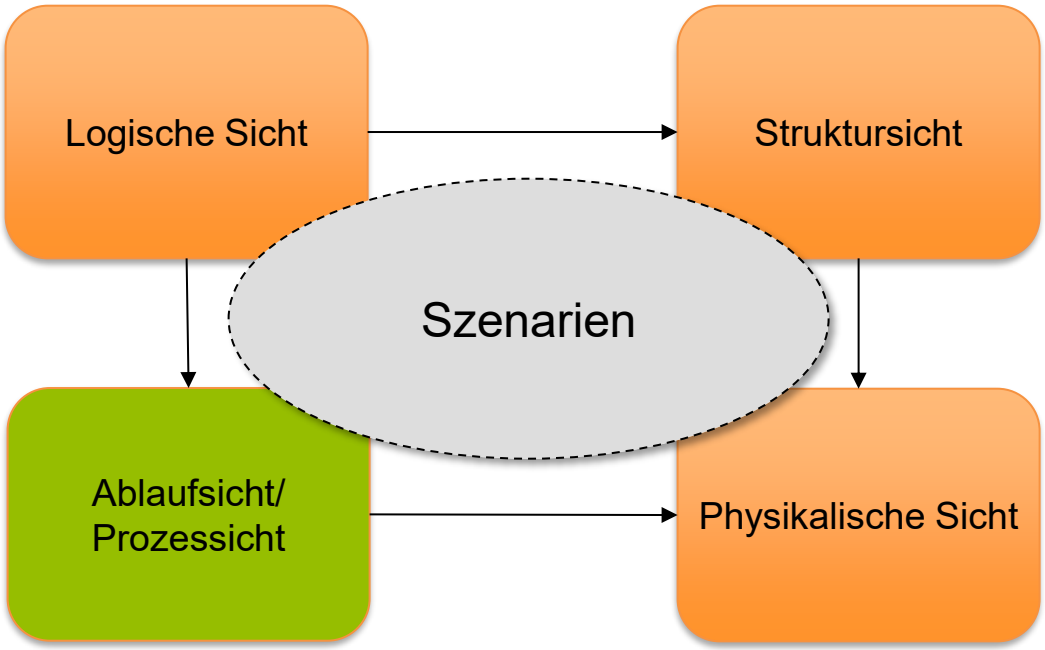
Version: 1.3

Letze Änderung: 13.03.2026

Die 4+1 Sichten der Softwarearchitektur

Es gibt keine allgemeine „Architekturdarstellung“.

Es müssen verschiedene Sichten (eines Systems) zu einer Gesamtarchitektur vereinigt werden



Philippe Kruchten, The 4+1 view model of architecture, IEEE Software, November 1995, 12(6), pp. 42-50

Prozesssicht Sicht der Architektur

Abbildung des Produktmodells auf ein Verarbeitungsmodell.

- Wie können eingehende Requests (z.B. Aufrufe einer Remote-Funktion) nebenläufig bearbeitet werden?
- Wie kann auf Benutzereingaben (Ereignisse) reagiert werden?

Betrachtet werden folgende Aspekte

- Teilmenge der nicht-funktionalen Anforderungen
  - Performance
  - Availability
- Nebenläufigkeit
- Koordination/Synchronisation

Prozesssicht Sicht der Architektur

Grundlegende Eigenschaften bzw. Erkenntnisse

- Die Prozesssicht weist wenig Ähnlichkeiten mit der Code-Struktur auf
- Besteht aus einer vernetzten Menge kommunizierender Objekte
- Die Element sind Komponenten welche nur zur Laufzeit von Bedeutung sind (Prozesse, Threads, EJB‘s, Servlets, DLL‘s, Queues, …)
- Die Interaktion der Elemente variiert sehr stark abhängig von der verwendeten Technologie
- Diese Sicht ist wichtig für die Betrachtung qualitativer Merkmale wie z.B. Performance und Zuverlässigkeit
- Beim Entwurf der Prozessarchitektur sollte eine möglichst strikte Trennung zwischen der Technologie sowie der Anwendungsarchitektur selbst stattfinden

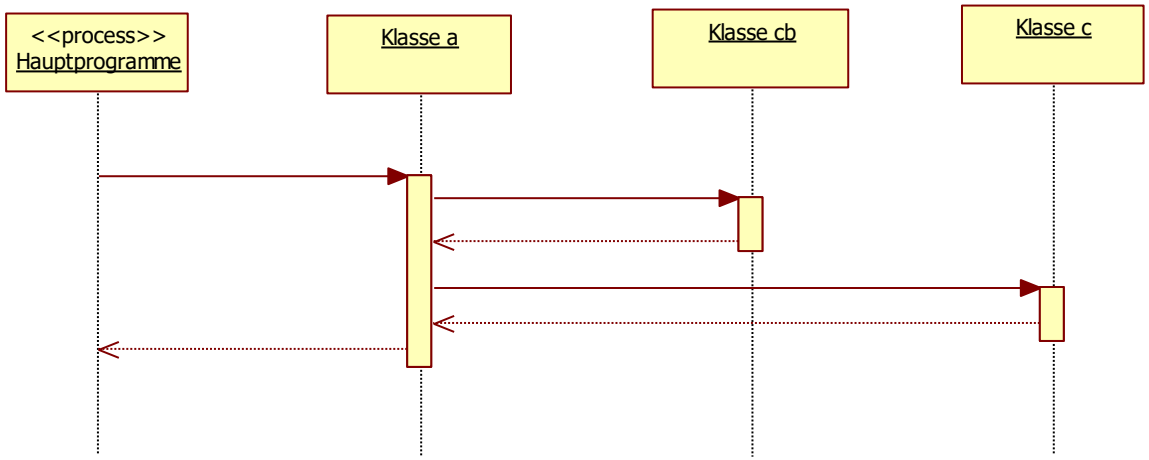
„Call-Return“

Prinzip:

- Es existiert ein Hauptprogramm welches Funktionen und Methoden der verschiedenen Subsystem aufruft
- Subsysteme dürfen selbst wiederum weitere Funktionen und Methoden aufrufen
- Diese Art der Strukturierung wird auch als „Top-Down-Architektur“ bezeichnet

„Call-Return“

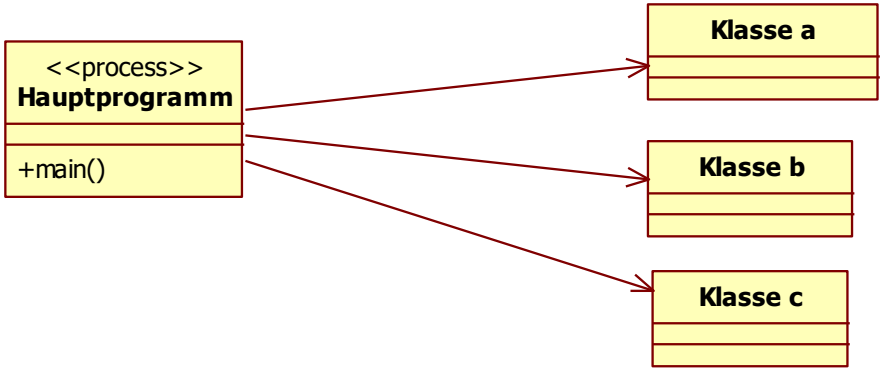
Dynamische Sichtweise



Das Hauptprogramm wird als Prozessgestartet und ruft verschiede Methoden auf

„Call-Return“

Statische Sichtweise



Anwendung & Vorteile:

- Einfache Programme mit klar definierter Struktur
- Keine Parallelität in der Verarbeitung
- Arbeitsschritte werden sequentiell ausgeführt

Ereignisgesteuerte Verarbeitung (Selective Broadcast)

Prinzip:

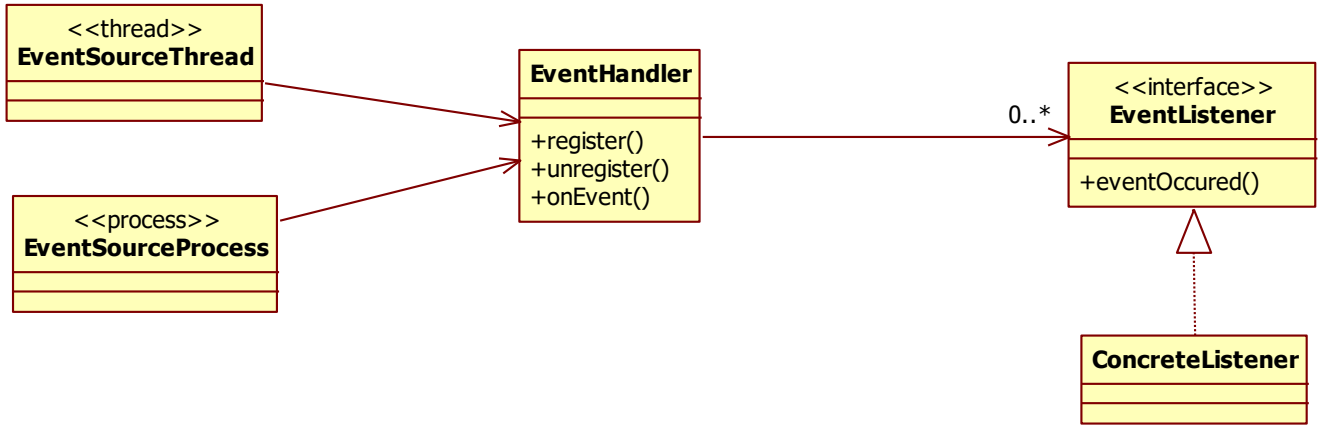
- Verwende ein Ereignisgesteuertes Modell anstelle von direkten Methodenaufrufen
- Komponenten können sich für den Empfang von Ereignissen registrieren.
- Tritt das Ereignis auf, können alle registrierten Komponenten von diesem Ereignis benachrichtig werden
- Auch bekannt als „Publish/Subscribe“ (im kleinen)

Anwendung:

- Pattern für graphische Oberflächen (z.B. Eclipse)
- Verteilte System können auf Basis dieses Muster lose gekoppelte werden

Ereignisgesteuerte Verarbeitung (Selective Broadcast)

Statische Sichtweise

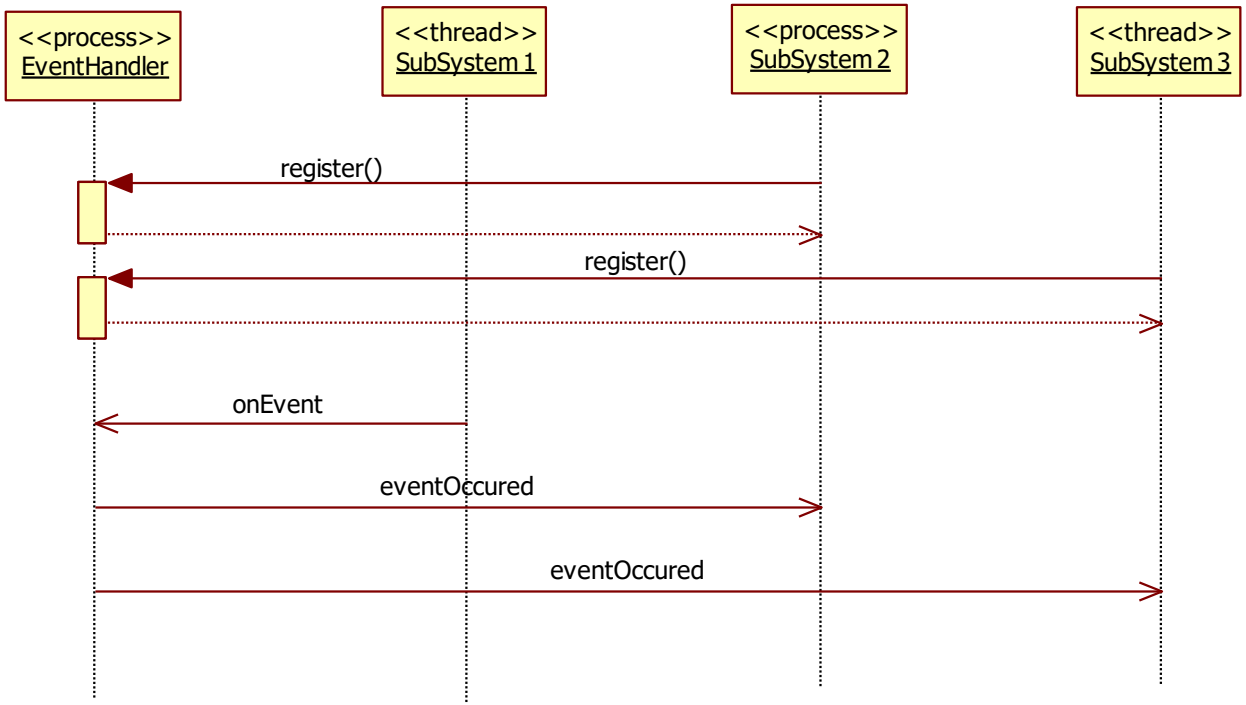


Idee:

- Wird ein Ereignis gemeldet (onEvent) wird dieses an die registrierten Listener weitergegeben
- Ein EventListener muss das Interesse an Events über die Methode „register“ bekannt geben

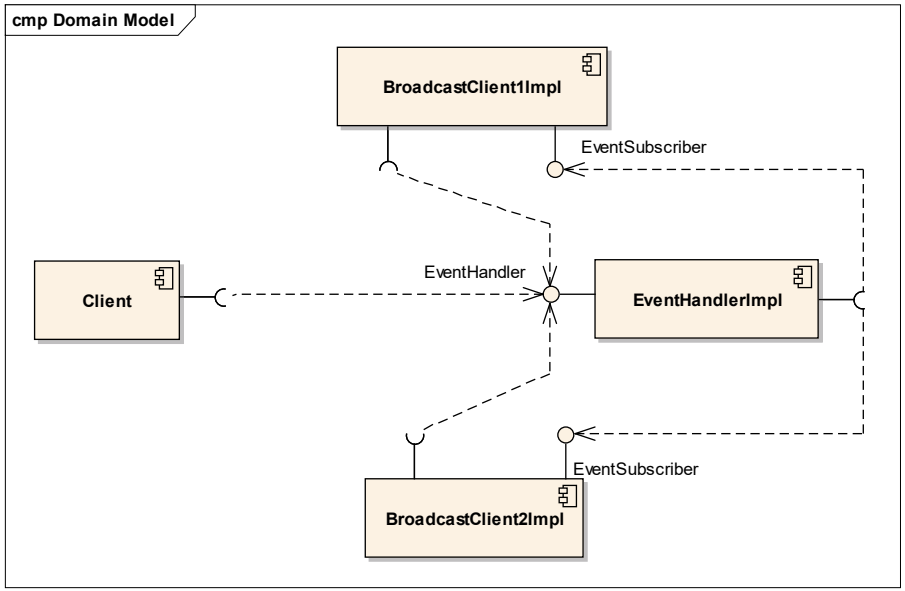
Ereignisgesteuerte Verarbeitung (Selective Broadcast)

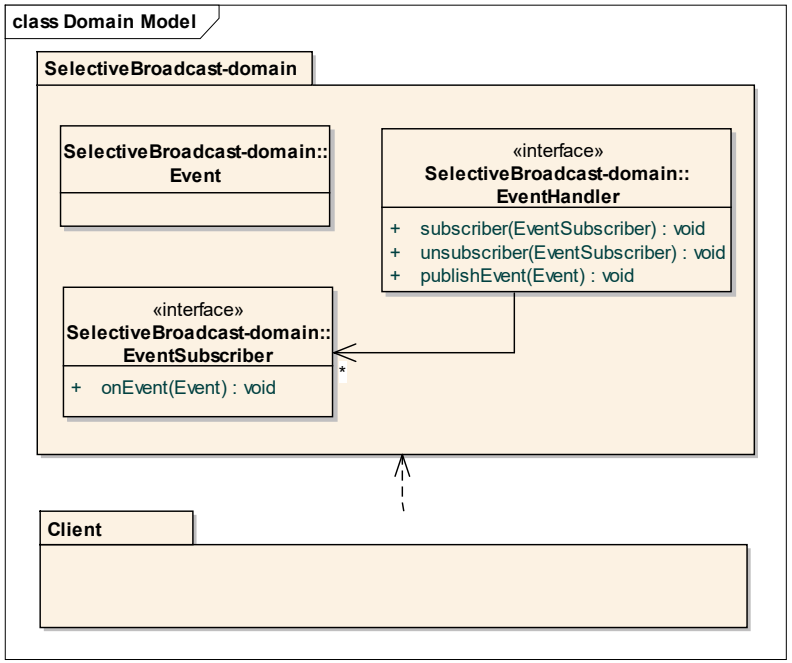
Dynamische Sichtweise



Ereignisgesteuerte Verarbeitung (Selective Broadcast)

Umsetzung als Komponentenarchitektur





Ereignisgesteuerte Verarbeitung (Selective Broadcast)

Vorteile:

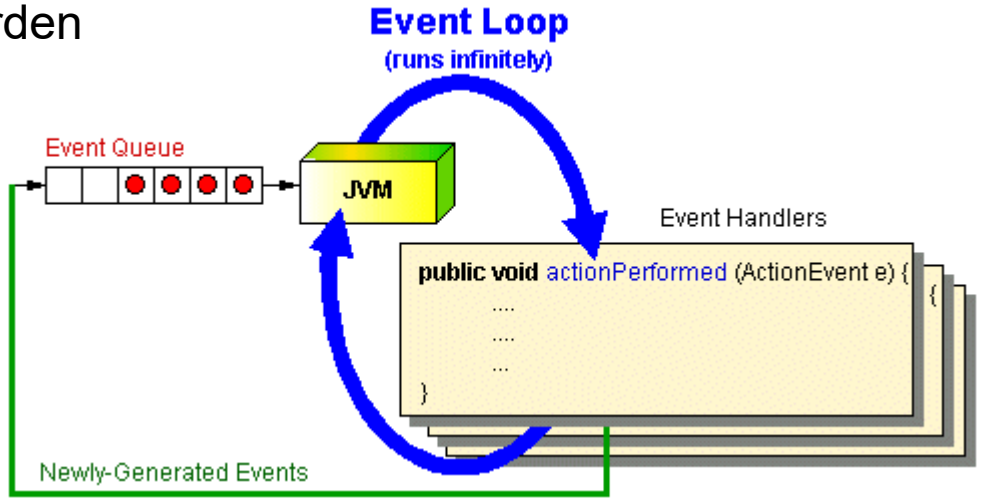
- Systeme welche auf diesem Muster basieren sind einfach zu erweitern
- Geringe Kopplung zwischen dem Kernsystem und den angebunden Komponenten
- Bestehende Komponenten können ausgetauscht werden

Probleme:

- Nach dem melden eines Events gibt die EventSource die Kontrolle über die Verarbeitung des Ereignisses ab (blockierende Aufrufe)
- Die Verarbeitungsreihenfolge über mehrere Listener ist nicht definiert
- Die Eventquelle weis nicht wann alle Events abgearbeitet sind

Ereignisgesteuerte Verarbeitung in grafischen Oberflächen

- Grafische Oberflächen arbeiten in der Regel ereignisgesteuert
- Ereignisse treten in unterschiedlicher Reihenfolge auf
- In der Regel existiert ein sog. Event-Loop. Dieser empfängt Ereignisse (wie z.B. eine Mausklick) und leitet das Ereignis an einen geeigneten Empfänger weiter
- Die Empfänger registrieren hierzu Callback-Interfaces welche bei einem Ereignis aufgerufen werden



Ereignissteuerung in grafischen Oberflächen (Android)

Reaktion auf einen Button-Klick

```
8   public class MainActivity extends AppCompatActivity {
9
10      @Override
11      protected void onCreate(Bundle savedInstanceState)
12          super.onCreate(savedInstanceState);
13          setContentView(R.layout.activity_main);
14
15          Button button = findViewById( R.id.button );
16
17          button.setOnClickListener( (event) -> {
18
19              button.setText( "Pressed" );
20
21          } );
22      }
23  }
```

Zugriff auf ein Objekt vom Typ „Button“
Repäsentiert den eigentlichen Button in Form eines Objektes mit vielfältigen Methoden und Attributen

Festlegen der Reaktion auf einen Button-Click

Ereignissteuerung in grafischen Oberflächen - Der Andorid-UI-Thread

- Wenn eine Anwendung gestartet wird, erstellt das System einen Thread für die Anwendung, der "main" genannt wird.
- Dieser Thread ist sehr wichtig, da er für die Weiterleitung von Ereignissen an die entsprechenden Widgets der Benutzeroberfläche zuständig ist, einschließlich Zeichenereignissen.
- Es ist auch fast immer der Thread, in dem Ihre Anwendung mit Komponenten aus dem Android UI-Toolkit interagiert
- Dieser Thread wird oft auch als UI-Thread bezeichnet.

https://developer.android.com/guide/components/processes-and-threads

Ereignissteuerung in grafischen Oberflächen der Andorid-UI-Thread

- Nachfolgend ist die Arbeitsweise des UI-Threads dargestellt

```
while( true ) {
    wait for an incoming Event ( e.g. touch, scroll gesture,...)
    dispatch Event to View-Object for processing
    repaint the user interface
}
```

- Der Thread wartet bis ein Ereignis (z.B. Touch) eintritt
- Danach wird dieses Ereignis an ein View-Object weiter gereicht welches darauf reagiert
- Falls in Verarbeitung des Events die Oberfläche modifiziert wurde, wird diese neu gezeichnet.
- Danach werden neue Events verarbeitet.
- Die Verarbeitung erfolgt nacheinander

Ereignissteuerung in grafischen Oberflächen - Der UI-Thread

- Wenn die Anwendung intensive Arbeit als Reaktion auf Benutzerinteraktionen ausführt, kann dieses Single-Thread-Modell zu einer schlechten Leistung führen, wenn die Anwendung nicht richtig implementiert ist.
- Wenn nämlich alles im UI-Thread passiert, blockiert die Durchführung langer Operationen wie Netzwerkzugriffe oder Datenbankabfragen die gesamte Benutzeroberfläche.
- Wenn der Thread blockiert ist, können keine Ereignisse, einschließlich Zeichenereignisse, versendet werden.
- Aus der Sicht des Benutzers scheint die Anwendung zu hängen.

Ereignissteuerung in grafischen Oberflächen - Persistenz von Daten

- Um zu verhindern, dass langlaufende Abfragen die Benutzeroberfläche blockieren, dürfen lang andauernde Operationen nur asynchron ausgeführt werden
- Diese Einschränkung bedeutet, dass z.B. SQL-Abfragen asynchron ausführen müssen.
- Liegt das Ergebnis der asynchronen Ausführung vor, muss diese mit der Oberfläche synchronisiert werden
- Das Toolkit stellt eine Methode bereit, um Aktionen and den UI-Thread zur Ausführung zu übergebenauszuführen:

```
Defintion der Methode:
     public final void runOnUiThread(Runnable action)

runOnUiThread( () -> {
    // Do update of ui-elements
} );
```

Ereignissteuerung in grafischen Oberflächen - Der Andorid-UI-Thread

- Der Ablauf einer Asynchronen Operation sieht prinzipiell wie folgt aus:

```
public void asyncOperation() {
    // do something long running
    // Modify the ui
    runOnUiThread( () -> {
        // Do update of ui-elements
    } );
}

Start asyncOperation in a background Thread (e.g. with the executor
service)
```

Ereignissteuerung in grafischen Oberflächen

Warum ist die ereignisgesteuerte Programmierung notwendig?

- Das (Android) UI-Toolkit ist nicht thread-sicher.
- Die Manipulationen an der Benutzeroberfläche dürfen nur vom UI-Thread aus erfolgen, andernfalls wäre eine komplexe Synchronisation notwendig
- Der Ansatz des UI-Threads beugt Race-Conditions vor
- Es gibt also einfach zwei Regeln für das ereignisgesteuerte Modell:
  - Den UI-Thread nicht blockieren
  - Do not access the Android UI toolkit from outside the UI thread
- Alle Aktionen welche die Oberfläche manipulieren müssen über den UI-Thread erfolgen.

Persistenz von Daten

- Ist der Rückgabetype „ListenableFuture“, dann wird das Ergebnis erst zu einem späteren Zeitpunkt zur Verfügung gestellt.
- Die Ausführung der eigentlichen Datenbankfunktion erfolgt asynchron in einem separaten Thread
- Dadurch wird verhindert dass die UI durch die Ausführung einer lang dauernden Datenbank blockiert wird.
- Um auf das Ergebnis zuzugreifen muss eine Callback-Funktion registriert werden welche aufgerufen wird wenn das Ergebnis vorliegt.

Persistenz von Daten

- Achtung: Die Ausführung des Listeners erfolgt asynchron in einem separaten Thread
- Aber: Änderungen an der UI dürfen nur in dem UI-Thread erfolgen. Dadurch ist sichergestellt dass keine Race-Conditions bei gleichzeitigen Zugriffen auf die Oberfläche erfolgt.

```
ListenableFuture<List<User>> resultFuture = dao.loadUsersByRegion( regions );

future.addListener( () -> {

            List<User> users = resultFuture.get();

            runOnUiThread( () -> {
                // Do update of ui-elements
            } );

      },
      Executors.newSingleThreadExecutor() );
```